



C Program Controls + Flow Charts

-controlling the program execution flow:
selection, repetition and branching-

PROGRAM CONTROL

Repetition: The `for` statement

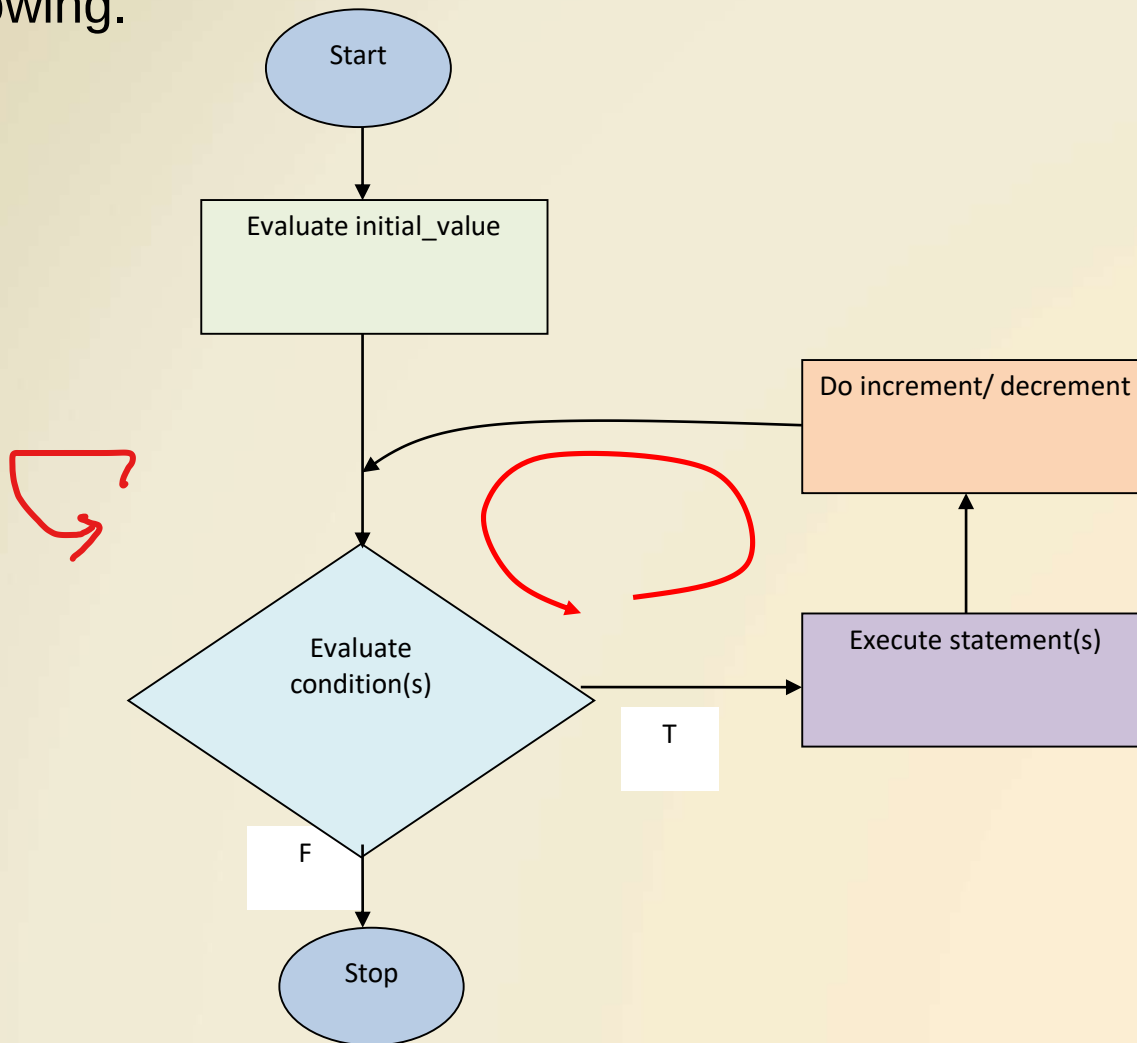
- Executes a code block for a certain number of times.
- The code block may have no statement, one statement or more.
- The `for` statement causes the `for` loop to be executed in a fixed number of times.
- The following is the `for` statement form,

```
for(initial_value;condition(s);increment/decrement)
    statement(s);
next_statement;
```

- `initial_value`, `condition(s)` and `increment/decrement` are any valid C expressions.
- The `statement(s)` may be a single or compound C statement (a block of code).
- When `for` statement is encountered during program execution, the following events occurs:
 1. The `initial_value` is evaluated e.g. `intNum = 1`.
 2. Then the `condition(s)` is evaluated, typically a relational expression.
 3. If `condition(s)` evaluates to FALSE (zero), the `for` statement terminates and execution passes to `next_statement`.
 4. If `condition(s)` evaluates as TRUE (non zero), the `statement(s)` is executed.
 5. Next, `increment/decrement` is executed, and execution returns to step no. 2 until `condition(s)` becomes FALSE.

PROGRAM CONTROL

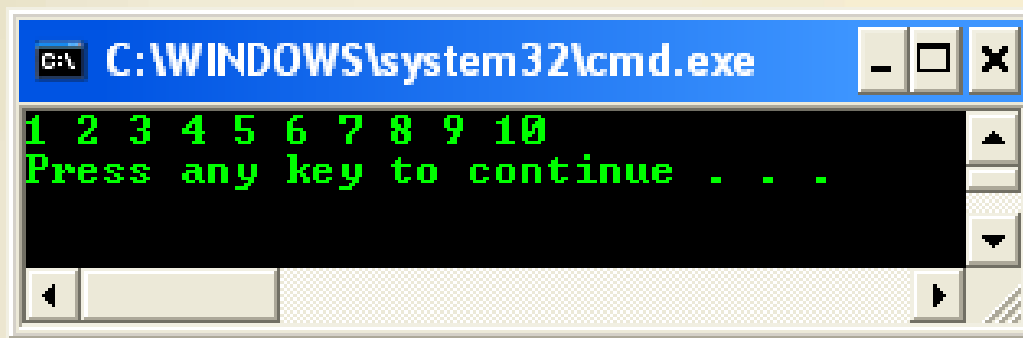
- The `for` loop flow chart should be something like the following.



PROGRAM CONTROL

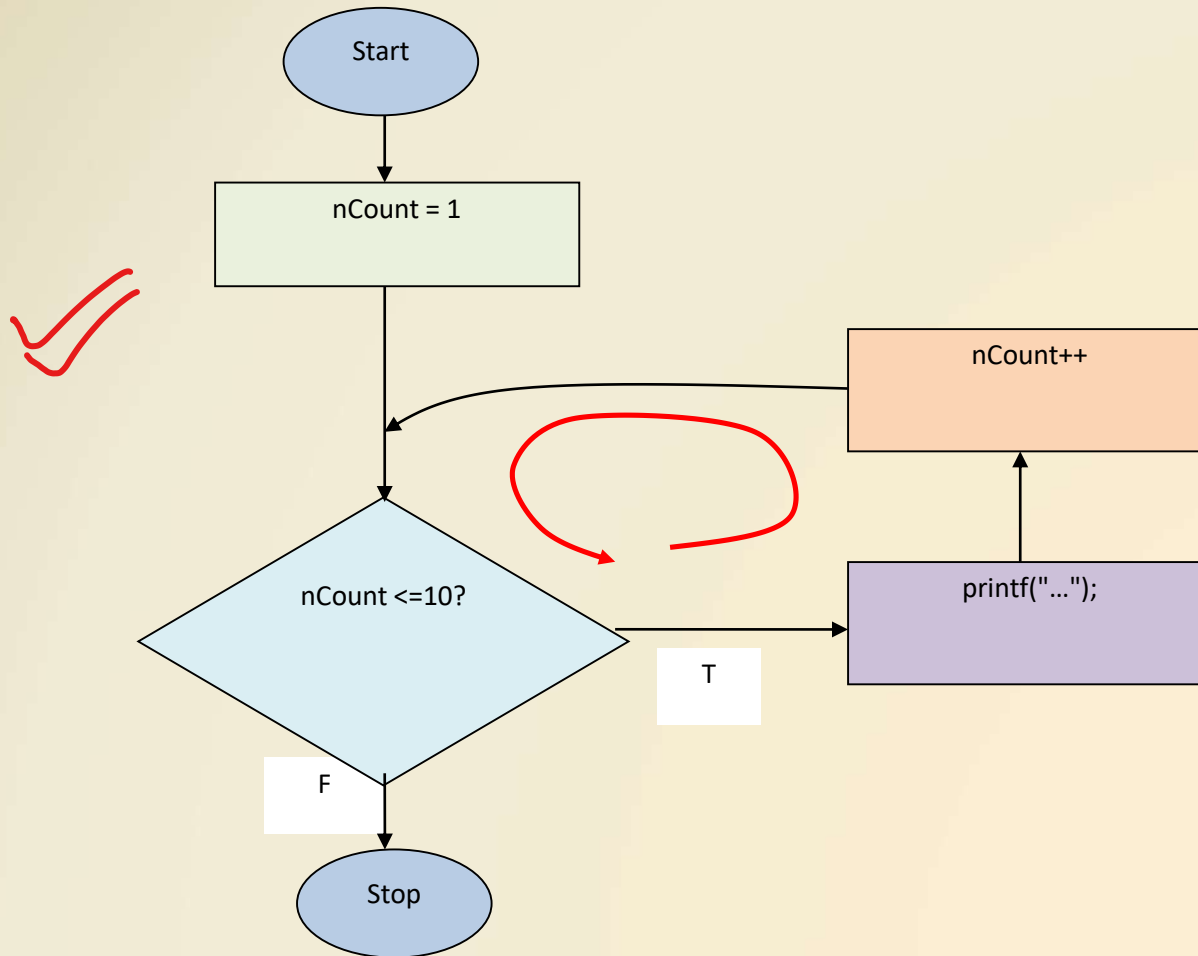
- A Simple `for` example, printing integer 1 to 10.

```
#include <stdio.h>
void main(void)
{
    int nCount;
    // display the numbers 1 to 10
    ✓ for(nCount = 1; nCount <= 10; nCount++)
        printf("%d ", nCount);
    printf("\n");
}
```



PROGRAM CONTROL

- Its flow chart...



PROGRAM CONTROL

- `for` loop is a very flexible construct.
- Can use the decrementing counter instead of incrementing. For example,

```
for (nCount = 100; nCount > 0; nCount--)
```

- Can use counter other than 1, for example 3,

```
for(nCount = 0; nCount < 1000; nCount += 3)
```

- `initial_value` can be omitted if the test variable has been initialized beforehand.
- However the semicolon must still be there. For example,

```
nCount=1;  
for( ; nCount < 1000; nCount ++)
```

PROGRAM CONTROL

- The `initial_value` can be any valid C expression, the expression is executed once when the `for` statement is first reached. For example,

```
nCount =1;  
for(printf("Now sorting the array..."); nCount < 1000;  
    nCount ++)
```

- The increment/decrement expression can be omitted as long as the counter variable is updated within the body of the `for` statement.
- The semicolon still must be included. For example,

```
for(nCount =0; nCount < 100; )  
printf("%d", nCount ++);
```

PROGRAM CONTROL

- The `condition(s)` expression that terminates the loop can be any valid C expression.
- As long as it evaluates as `TRUE` (non zero), the `for` statement continues to execute.
- Logical operators can be used to construct more complex `condition(s)` expressions. For example,

```
for(nCount =0; nCount < 1000 && name[nCount] != 0; nCount ++)  
    printf("%d", name[nCount]);  
for(nCount = 0; nCount < 1000 && list[nCount];)  
    printf("%d", list[nCount ++]);
```

- Note: The `for` statement(s) and arrays are closely related, so it is difficult to define one without explaining the other (will be discussed in another Chapter).

PROGRAM CONTROL

- The `for` statement(s) can be followed by a null (empty) statement, so that task is done in the `for` loop itself.
- Null statement consists of a semicolon alone on a line. For example,

```
for(count = 0; count < 20000; count++)  
    ;
```



- This statement provides a pause (delay) of 20,000 milliseconds.

PROGRAM CONTROL

- An expression can be created by separating two sub expressions with the comma operator, and are evaluated (in left-to-right order), and the entire expression evaluates to the value of the right sub expression.
- Each part of the `for` statement can be made to perform multiple duties. For example,

"We have two arrays with 1000 elements each, named `a[ ]` and `b[ ]`. Then we want to copy the contents of `a[ ]` to `b[ ]` in the reverse order, so, after the copy operation, the array content should be..."

`b[0], b[1], b[2],...` and `a[999], a[998], a[997],...` and so on.

- sample coding is,

```
for(iRow = 0, jColumn = 999; iRow < 1000; iRow ++, jColumn--)  
    b[jColumn] = a[iRow];
```



PROGRAM CONTROL

- Another examples of the `for` statements,

```
nSum = 0;
for(iRow = 1; iRow <=20; iRow++)
    nSum = nSum + iRow;
printf("\n Sum of the first 20 natural numbers = ");
printf("Sum = %d", nSum);
```

- The above program segment will compute and display the sum of the first 20 natural numbers.
- The above example can be re-written as,

```
for(iNum = 1, nSum = 0; iNum <= 20; iNum++)
    nSum = nSum + iNum;
printf("Sum of the first 20 natural numbers = %d", nSum);
```

- Take note that the initialization part has two statements separated by a comma (,).

PROGRAM CONTROL

- Another example,

```
for(iNum = 2, nSum=0, nSum2 = 0; iNum <= 20; iNum = iNum + 2)
{
    nSum = nSum + iNum;
    nSum2 = nSum2 + iNum * iNum;
}
printf("Sum of the first 20 even natural numbers = %d\n", nSum);
printf("Sum of the square for the first 20 even natural numbers = %d", nSum2);
```

- In this example, the `for` statement is a compound or block statement.
- Note that, the initial value in the initialization part doesn't have to be zero and the increment value unnecessarily needs to be 1.

PROGRAM CONTROL

- We can also create an infinite or never-ending loop by omitting all the expressions or by using a non-zero constant for condition(s) as shown in the following two code snippets,

```
for( ; ; )  
    printf("This is an infinite loop\n");
```

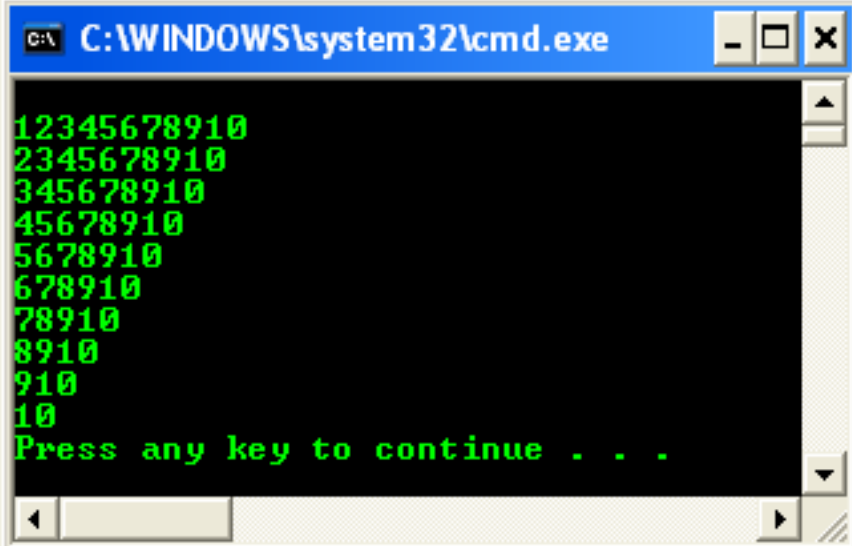
- or

```
for( ; 1 ; )  
    printf("This is an infinite loop\n");
```

- In both cases, the message "This is an infinite loop" will be printed repeatedly, indefinitely.
- All the repetition constructs discussed so far can be nested to any degree.

PROGRAM CONTROL

The nested for example



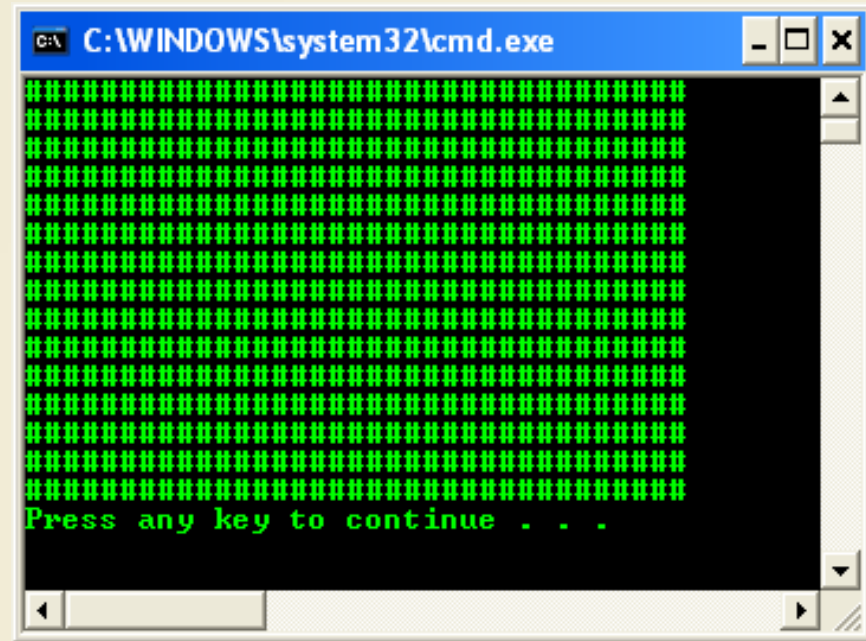
```
C:\WINDOWS\system32\cmd.exe

12345678910
2345678910
345678910
45678910
5678910
678910
78910
8910
910
10
Press any key to continue . . .
```

- The program has two `for` loops. The loop index `iRow` for the outer (first) loop runs from 1 to 10 and for each value of `iRow`, the loop index `jColumn` for the inner loop runs from `iRow + 1` to 10.
- Note that for the last value of `iRow` (i.e. 10), the inner loop is not executed at all because the starting value of `jColumn` is 2 and the expression `jColumn < 11` yields the value `false` (`jColumn = 11`).

PROGRAM CONTROL

Another nested for example



1. In the first `for` loop, the initialization is skipped because the initial value of `row`, 10 has been initialized; this `for` loop is executed until the `row` is 1 (`row > 0`).
2. For every `row` value, the inner `for` loop will be executed until `col = 1` (`col > 0`).
3. So the external `for` loop will print the `row` and the internal `for` loop will print the `column` so we got a rectangle of #.

PROGRAM CONTROL

Repetition: The `while` loop

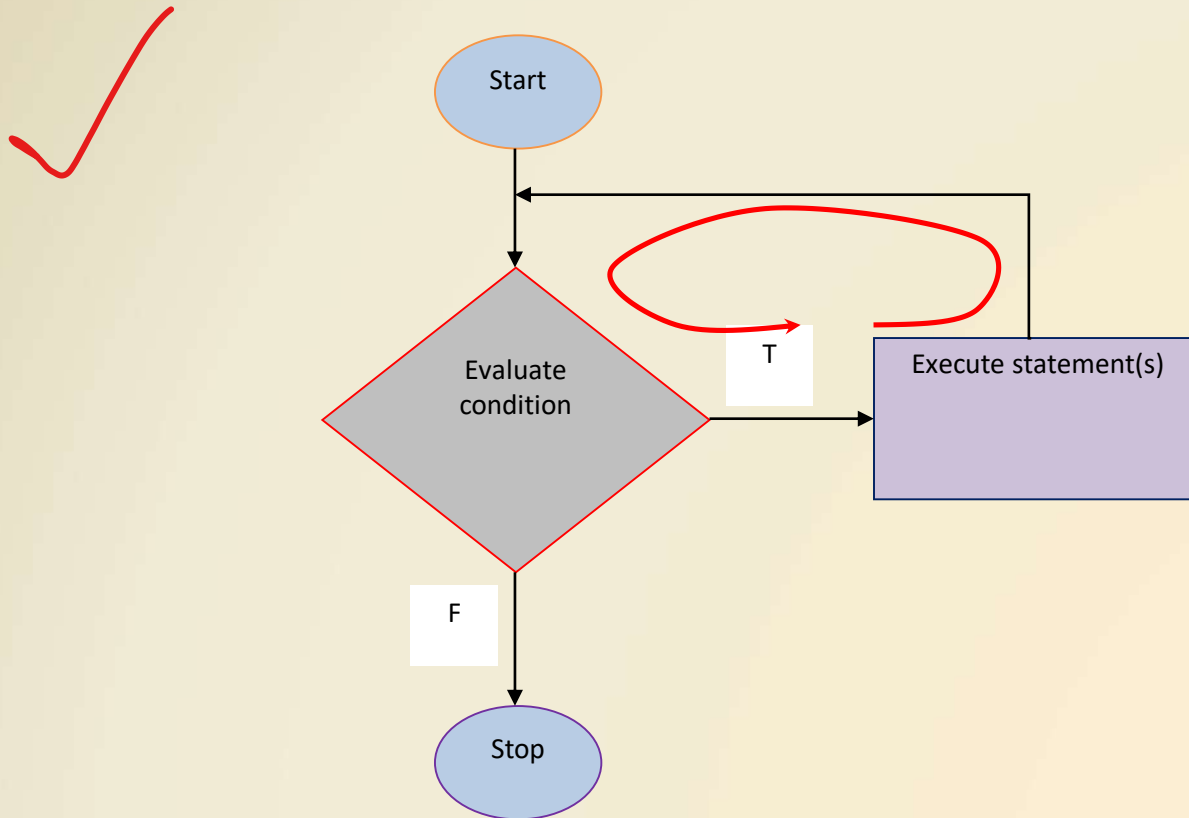
- Executes a block of statements as long as a specified condition is `TRUE`.
- The general `while` loop construct,

```
while (condition)
    statement(s);
next_statement;
```

- The `(condition)` may be any valid C expression.
- The `statement(s)` may be either a single or a compound (a block of code) C statement.
- When `while` statement encountered, the following events occur:
 1. The `(condition)` is evaluated.
 2. If `(condition)` evaluates to `FALSE` (zero), the `while` loop terminates and execution passes to the `next_statement`.
 3. If `(condition)` evaluates as `TRUE` (non zero), the C `statement(s)` is executed.
 4. Then, the execution returns to step number 1 until condition becomes `FALSE`.

PROGRAM CONTROL

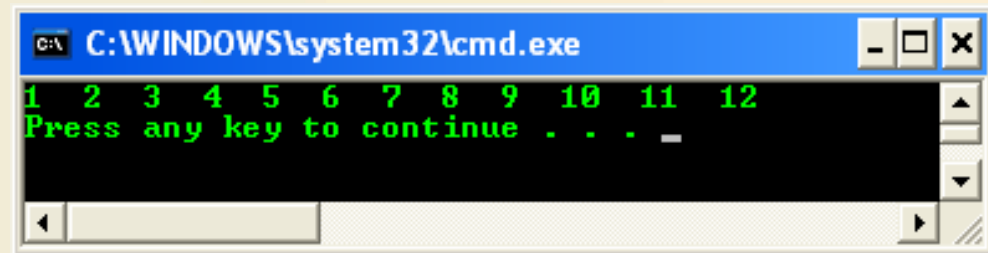
- The `while` statement flow chart is shown below.



PROGRAM CONTROL

- A simple example

```
// simple while loop example
#include <stdio.h>
int main(void)
{
    int nCalculate = 1;
    // set the while condition
    while(nCalculate <= 12)
    {
        // print
        printf("%d ", nCalculate);
        // increment by 1, repeats
        nCalculate++;
    }
    // a newline
    printf("\n");
    return 0;
}
```



PROGRAM CONTROL

- The same task that can be performed using the `for` statement.
- But, `while` statement does not contain an initialization section, the program must explicitly initialize any variables beforehand.
- As conclusion, `while` statement is essentially a `for` statement without the initialization and increment components.
- The syntax comparison between `for` and `while`,

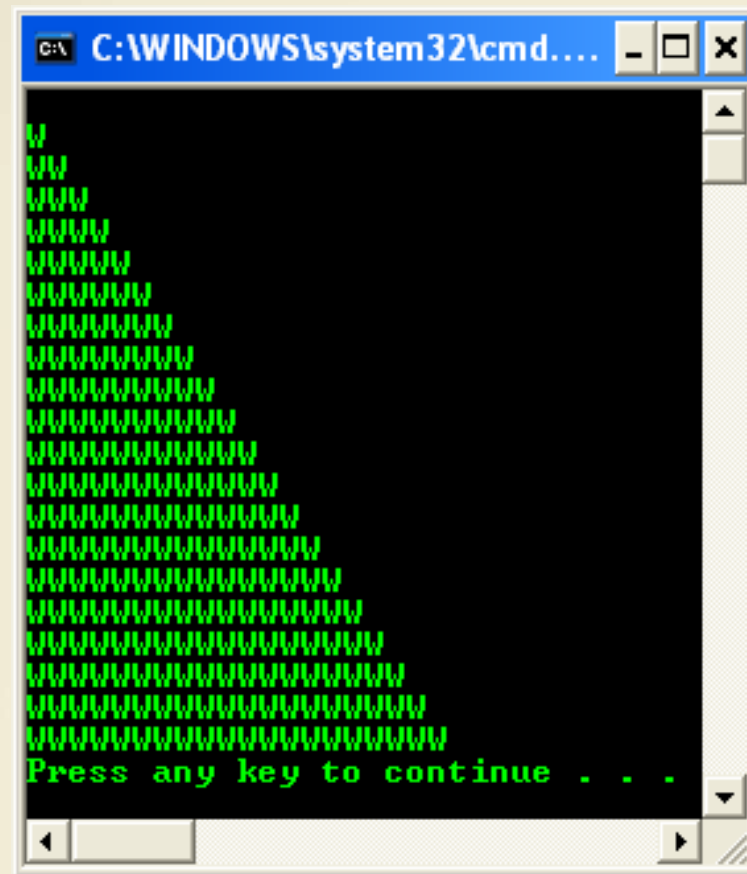
```
for ( ; condition; )
```

vs

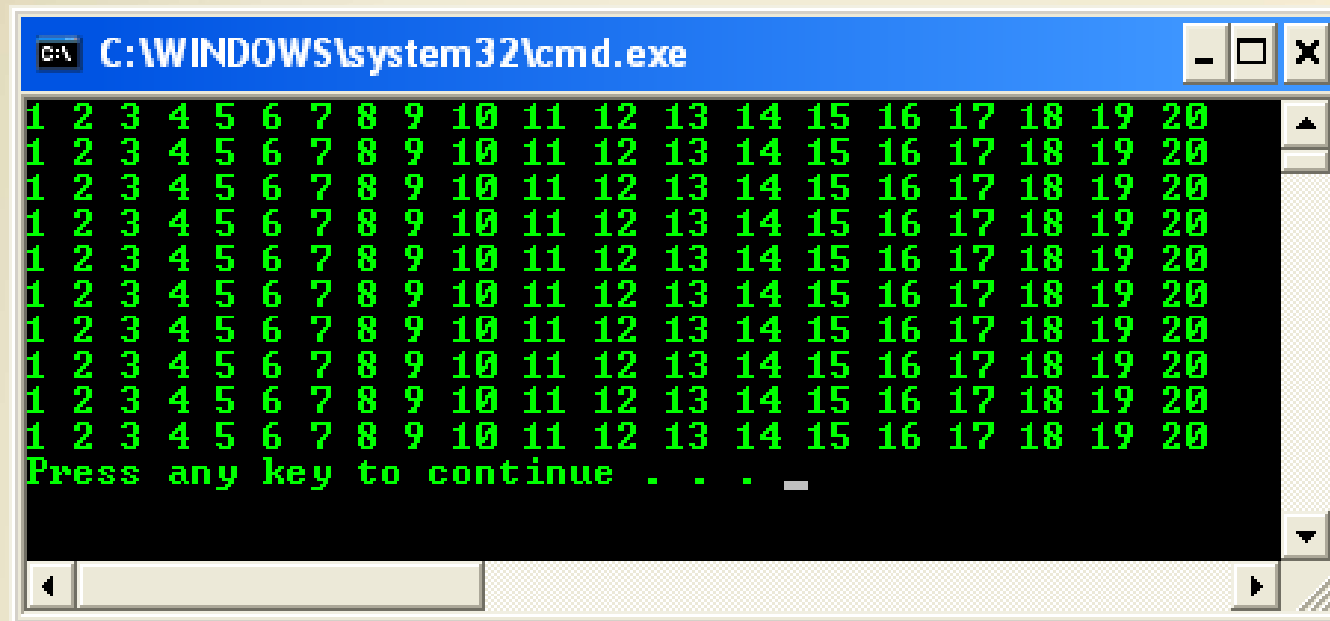
```
while (condition)
```

PROGRAM CONTROL

- Just like for and if statements, while statements can also be nested.



PROGRAM CONTROL



A screenshot of a Windows command prompt window. The title bar is blue and contains the text "C:\WINDOWS\system32\cmd.exe" and standard window control buttons (minimize, maximize, close). The command prompt area has a black background with green text. It displays ten rows of numbers from 1 to 20, with each row containing 20 numbers. Below the numbers, it says "Press any key to continue . . . _". The window has a scroll bar on the right and a status bar at the bottom.

```
C:\WINDOWS\system32\cmd.exe  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20  
Press any key to continue . . . _
```

PROGRAM CONTROL

Repetition: The `do-while` loop

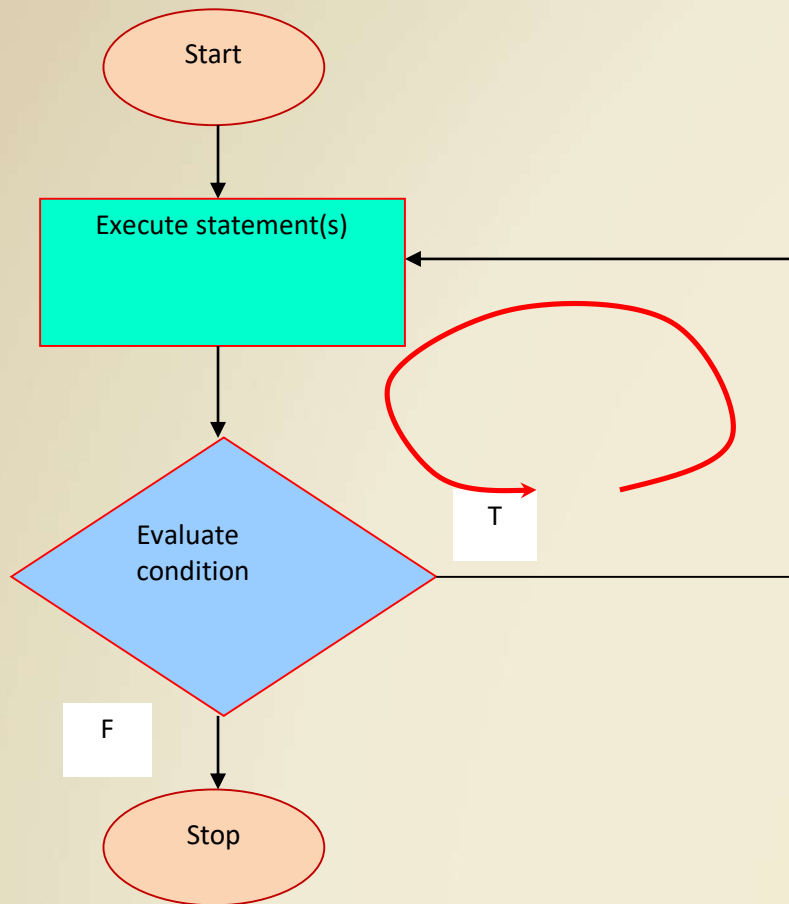
- Executes a block of statements as long as a specified condition is true at least once.
- Test the condition at the end of the loop rather than at the beginning, as demonstrated by the `for` and `while` loops.
- The `do-while` loop construct is,

```
do
    statement(s);
while (condition)
next_statement;
```

- `(condition)` can be any valid C expression.
- `statement(s)` can be either a single or compound (a block of code) C statement.
- When the program encounter the `do-while` loop, the following events occur:
 1. The `statement(s)` are executed.
 2. The `(condition)` is evaluated. If it is `TRUE`, execution returns to step number 1. If it is `FALSE`, the loop terminates and the `next_statement` is executed.
 3. This means the `statement(s)` in the `do-while` will be executed at least once.

PROGRAM CONTROL

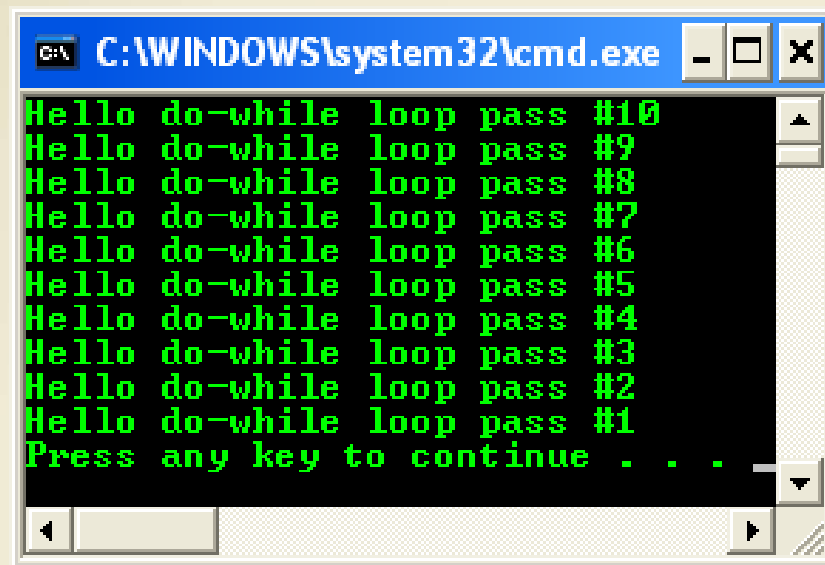
- A flow chart for the `do-while` loop



- The `statement(s)` are always executed at least once.
- `for` and `while` loops evaluate the condition at the start of the loop, so the associated statements are not executed if the condition is initially FALSE.

PROGRAM CONTROL

- [The do-while program example](#)



A screenshot of a Windows command prompt window. The title bar shows the path `C:\WINDOWS\system32\cmd.exe`. The window contains the following text in green on a black background:

```
Hello do-while loop pass #10  
Hello do-while loop pass #9  
Hello do-while loop pass #8  
Hello do-while loop pass #7  
Hello do-while loop pass #6  
Hello do-while loop pass #5  
Hello do-while loop pass #4  
Hello do-while loop pass #3  
Hello do-while loop pass #2  
Hello do-while loop pass #1  
Press any key to continue . . .
```

The text is displayed in a monospaced font. The window has standard Windows controls (minimize, maximize, close) in the title bar and a scrollbar on the right side.

PROGRAM CONTROL

Other Program Controls

`continue` keyword

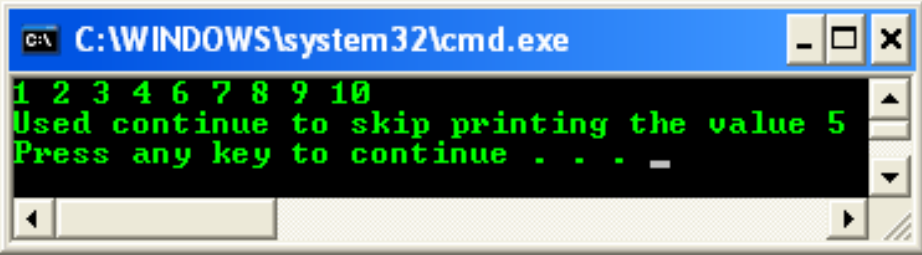
- `continue` keyword forces the next iteration to take place immediately, skipping any instructions that may follow it.
- The `continue` statement can only be used inside a loop (`for`, `do-while` and `while`) and not inside a `switch-case` selection.
- When executed, it transfers control to the condition (the expression part) in a `while` or `do-while` loop, and to the increment expression in a `for` loop.
- Unlike the `break` statement, `continue` does not force the termination of a loop, it merely transfers control to the next iteration.

PROGRAM CONTROL

- Consider the [following continue keyword example](#)

```
// using the continue in for structure
#include <stdio.h>

int main(void)
{
    int iNum;
    for(iNum = 1; iNum <= 10; iNum++)
    {
        // skip remaining code in loop only if iNum == 5
        if(iNum == 5)
            continue;
        printf("%d ", iNum);
    }
    printf("\nUsed continue to skip printing the value 5\n");
    return 0;
}
```



A screenshot of a Windows command prompt window titled "C:\WINDOWS\system32\cmd.exe". The window has a blue title bar and standard Windows window controls (minimize, maximize, close). The command prompt shows the output of the C program: "1 2 3 4 6 7 8 9 10" on the first line, "Used continue to skip printing the value 5" on the second line, and "Press any key to continue . . ." on the third line. The text is green on a black background. A cursor is visible at the end of the third line.

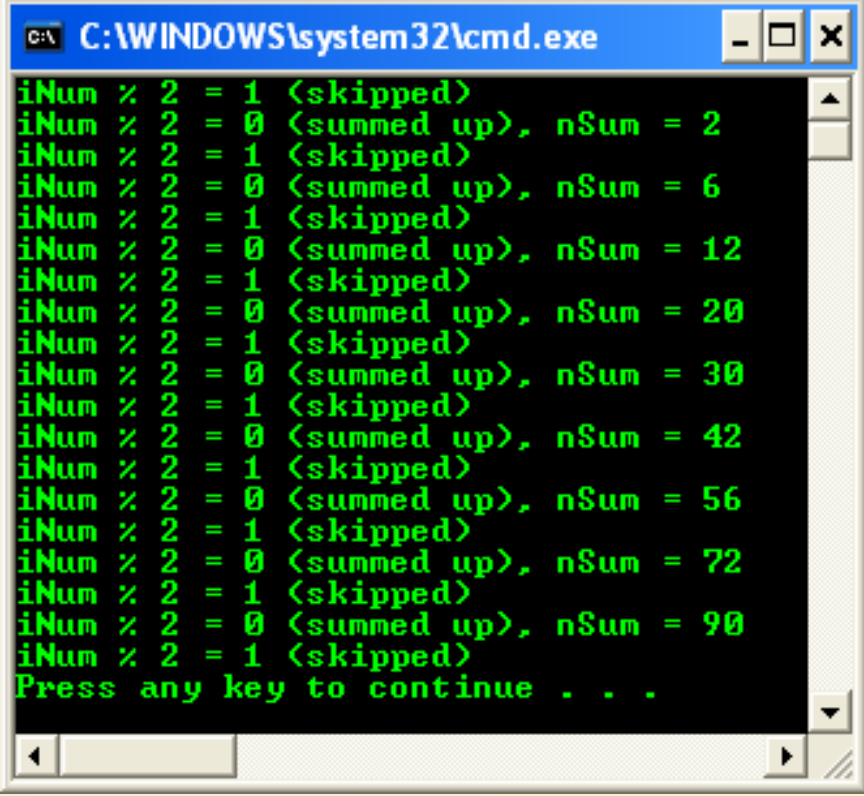
PROGRAM CONTROL

- Next consider the [following continue keyword example](#),

```
#include <stdio.h>

int main(void)
{
    int iNum, nSum;
    for(iNum=1, nSum=0; iNum<20; iNum++)
    {
        // test value, 0 or non-zero
        if (iNum % 2)
        {
            printf("iNum %% 2 = %d (skipped)\n", iNum % 2);
            // executed if the test value is non-zero
            // and repeat the for statement
            continue;
        }
        // executed if the test value is zero and repeat the for statement
        nSum = nSum + iNum;
        printf("iNum %% 2 = %d (summed up), nSum = %d \n", iNum % 2, nSum);
    }
    return 0;
}
```

PROGRAM CONTROL

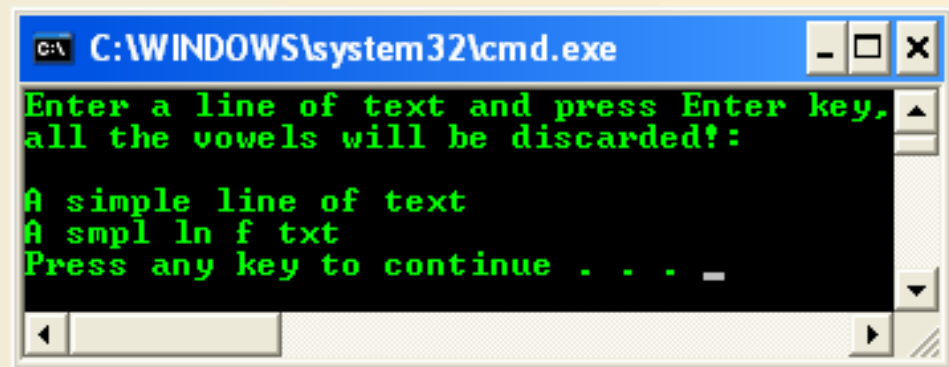


```
C:\WINDOWS\system32\cmd.exe
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 2
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 6
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 12
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 20
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 30
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 42
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 56
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 72
iNum % 2 = 1 <skipped>
iNum % 2 = 0 <summed up>, nSum = 90
iNum % 2 = 1 <skipped>
Press any key to continue . . .
```

- This loop sums up the even numbers 2, 4, 6, ... and stores the value in the `nSum` variable.
- If the expression `iNum % 2` (the remainder when `iNum` is divided by 2) yields a non-zero value (i.e., if `iNum` is odd), the `continue` statement is executed and the iteration repeated (`iNum` incremented and tested).

PROGRAM CONTROL

- If it yields a zero value (i.e., if `iNum` is even), the statement `nSum = nSum + iNum;` is executed and the iteration continued.
- When a `continue` statement executes, the next iteration of the enclosing loop begins.
- The enclosing loop means the statements between the `continue` statement and the end of the loop are not executed.
- [Try another continue example](#)



```
C:\WINDOWS\system32\cmd.exe

Enter a line of text and press Enter key,
all the vowels will be discarded!:

A simple line of text
A smpl ln f txt
Press any key to continue . . .
```

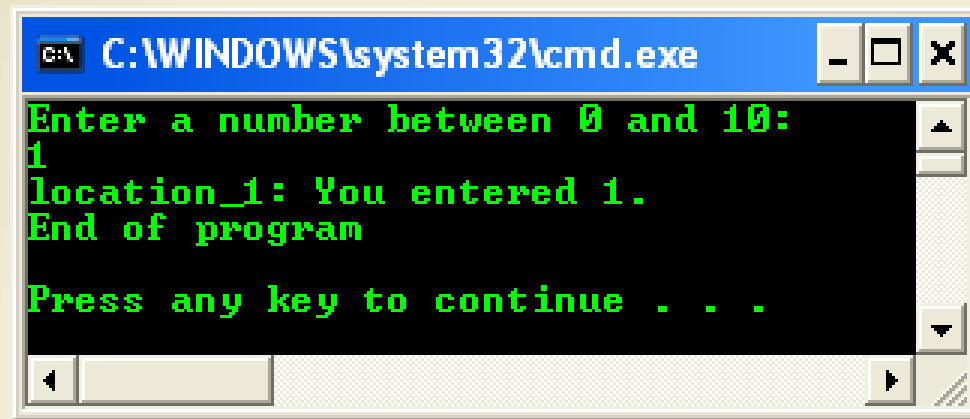
PROGRAM CONTROL

`goto` keyword

- The `goto` statement is one of C unconditional jump or branching.
- When program execution encounters a `goto` statement, execution immediately jumps, or branches, to the location specified by the `goto` statement.
- The statement is unconditional because execution always branches when a `goto` statement is came across, the branching does not depend on any condition.
- A `goto` statement and its target label must be located in the same function, although they can be in different blocks.
- Use `goto` to transfer execution both into and out of loop.
- However, using `goto` statement strongly not recommended.
- Always use other C branching statements.
- When program execution branches with a `goto` statement, no record is kept of where the execution is coming from.

PROGRAM CONTROL

- [Try the following C goto keyword program example](#)



A screenshot of a Windows command prompt window. The title bar is blue and contains the text "C:\WINDOWS\system32\cmd.exe" along with standard window control buttons (minimize, maximize, close). The command prompt has a black background with green text. The text displayed is: "Enter a number between 0 and 10:", followed by the user input "1" on the next line. The next line shows "location_1: You entered 1." and the following line is "End of program". The final line of text is "Press any key to continue . . .". A small white box is visible at the bottom left of the window, and a scrollbar is on the right side.

```
C:\WINDOWS\system32\cmd.exe
Enter a number between 0 and 10:
1
location_1: You entered 1.
End of program
Press any key to continue . . .
```

PROGRAM CONTROL

`exit()` function

- `exit()` function normally used when a program want to terminate at any time.
- The `exit()` function terminates program execution and returns control to the Operating System.
- The syntax of the `exit()` function is,

```
exit(status);
```

Status	Description
0 (zero)	The program terminated normally.
1 (or non-zero)	Indicates that the program terminated with some sort of error. The return value is usually ignored.

PROGRAM CONTROL

- We must include the header file [stdlib.h](#) (`cstdlib` if used in C++ code).
- This header file also defines two symbolic constants for use as arguments to the `exit()` function, such as,

```
#define EXIT_SUCCESS 0
#define EXIT_FAILURE 1
```

- Then we can call the function like the following,

```
exit(EXIT_SUCCESS);
```

Or

```
exit(EXIT_FAILURE);
```

PROGRAM CONTROL

`atexit()` function

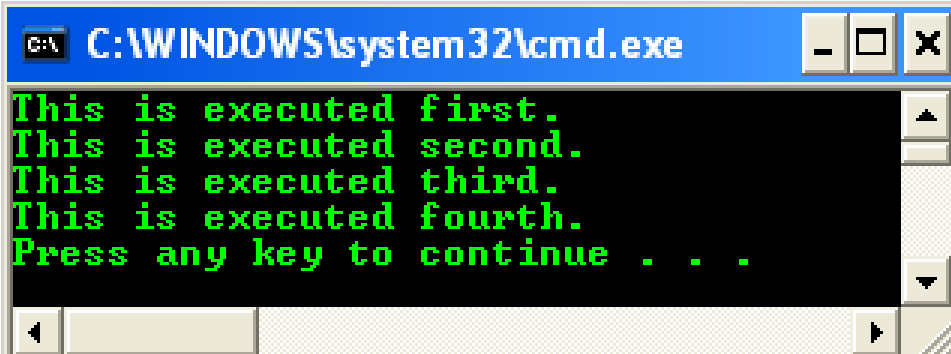
- Used to specify, or register, one or more functions that are automatically executed when the program terminates.
- Exit-processing function that executes prior to program termination
- These functions are executed on a last-in, first-out (LIFO) basis, the last function registered is the first function executed.
- When all functions registered by `atexit()` executed, the program terminates and returns control to the OS.
- The prototype of the `atexit()` function is located in the `stdlib.h` and the syntax is,

```
int  atexit(void(*funct) (void));
```

- where `funct` is the function to be called.

PROGRAM CONTROL

- `atexit()` function takes a function pointer as its argument and functions with `atexit()` must have a return type of `void`.
- The functions passed to `atexit()` cannot take parameters.
- `atexit()` uses the heap (instead of stack) to hold the registered functions.
- The following program pushes three functions onto the stack of functions to be executed when `atexit()` is called.
- When the program exits, these programs are executed on a last in, first out basis.



A screenshot of a Windows command prompt window. The title bar is blue and contains the text "C:\WINDOWS\system32\cmd.exe". The window has standard minimize, maximize, and close buttons. The command prompt area has a black background with green text. The text displayed is: "This is executed first.", "This is executed second.", "This is executed third.", "This is executed fourth.", and "Press any key to continue . . .". The text is left-aligned. At the bottom of the window, there is a white input field and a scroll bar on the right side.

```
C:\WINDOWS\system32\cmd.exe  
This is executed first.  
This is executed second.  
This is executed third.  
This is executed fourth.  
Press any key to continue . . .
```

End of C Program Looping

C has three loop statements: the while, the for, and the do...while. The first two are pretest loops, and the third is a post-test loop. We can use all of them for event-controlled and counter-controlled loops.

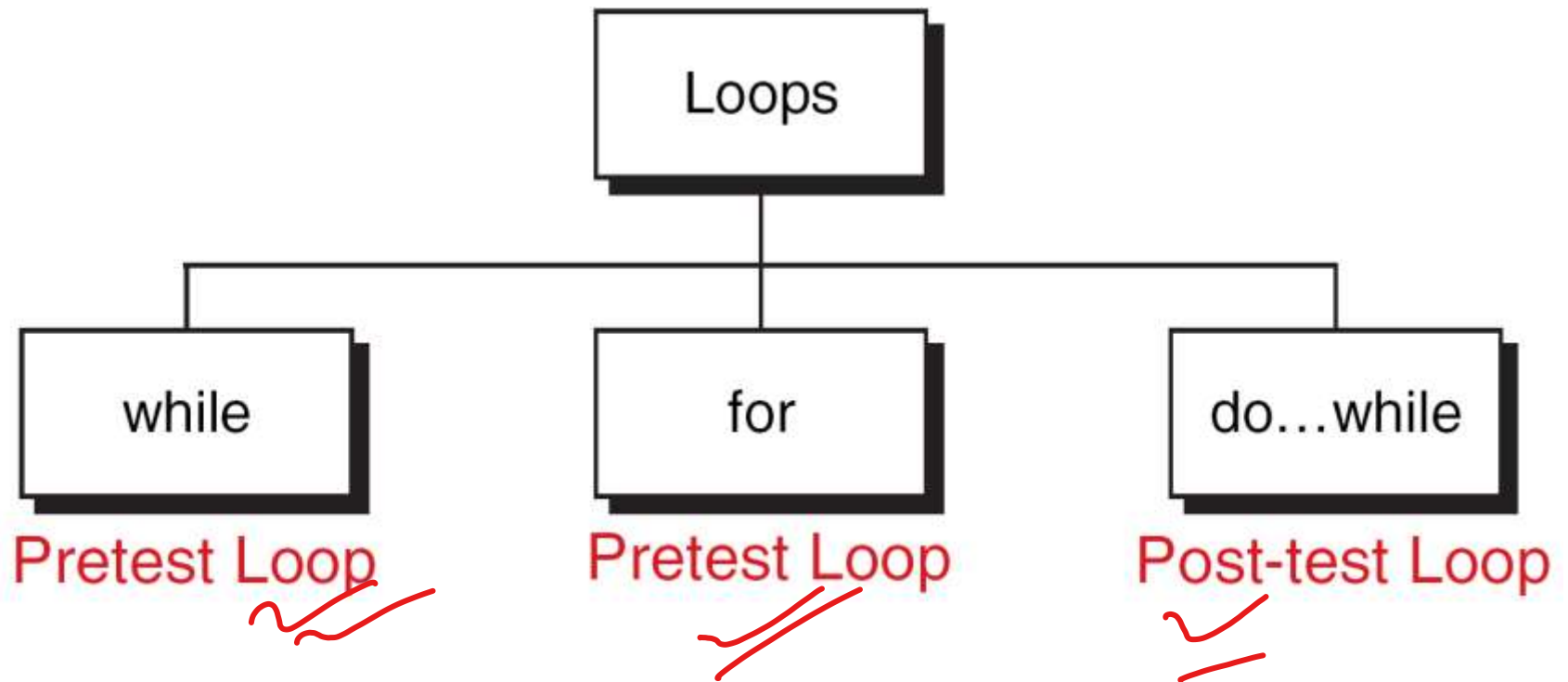
Topics discussed in this section:

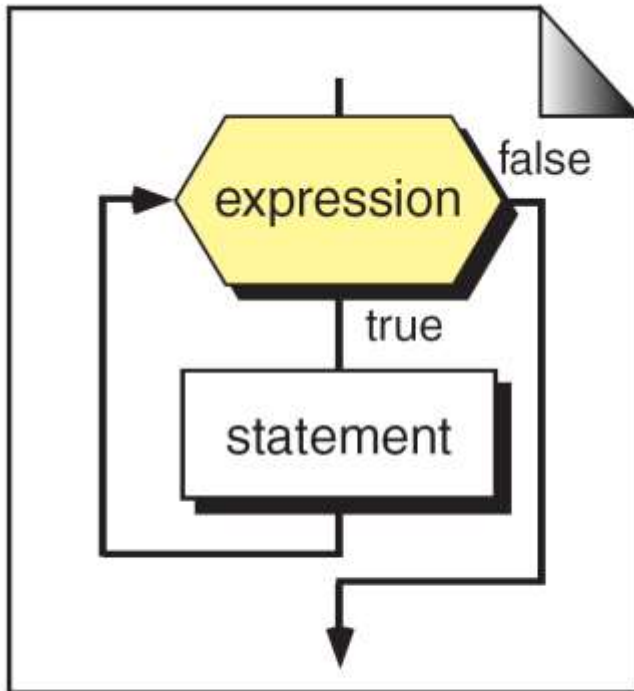
The while Loop

The for Loop

The do...while Loop

The Comma Expression

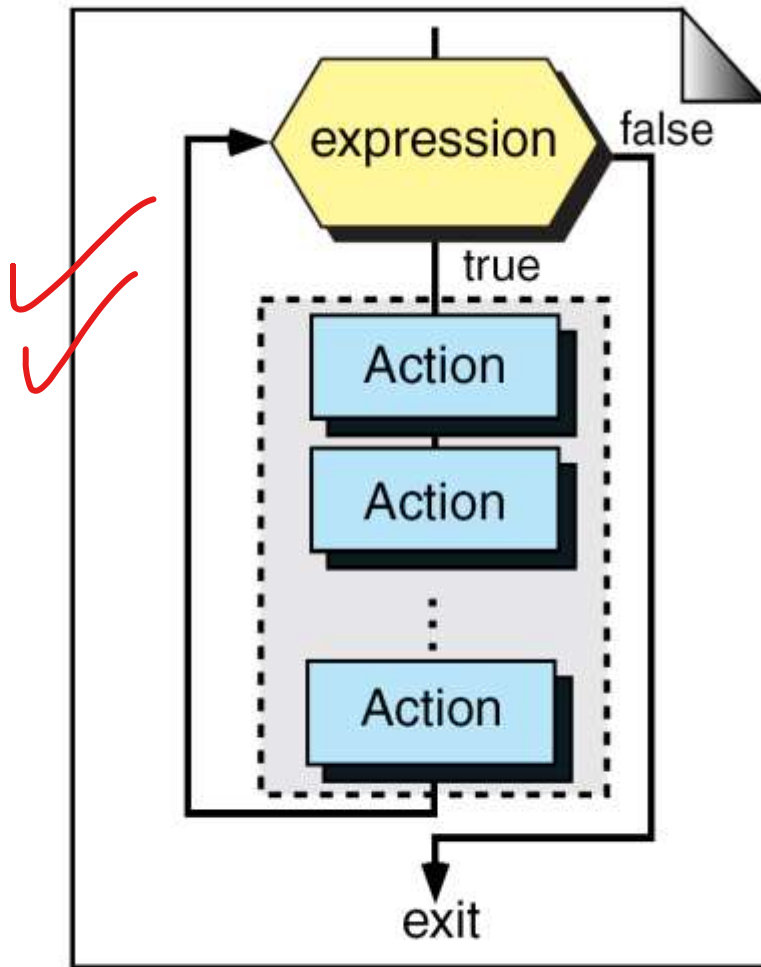




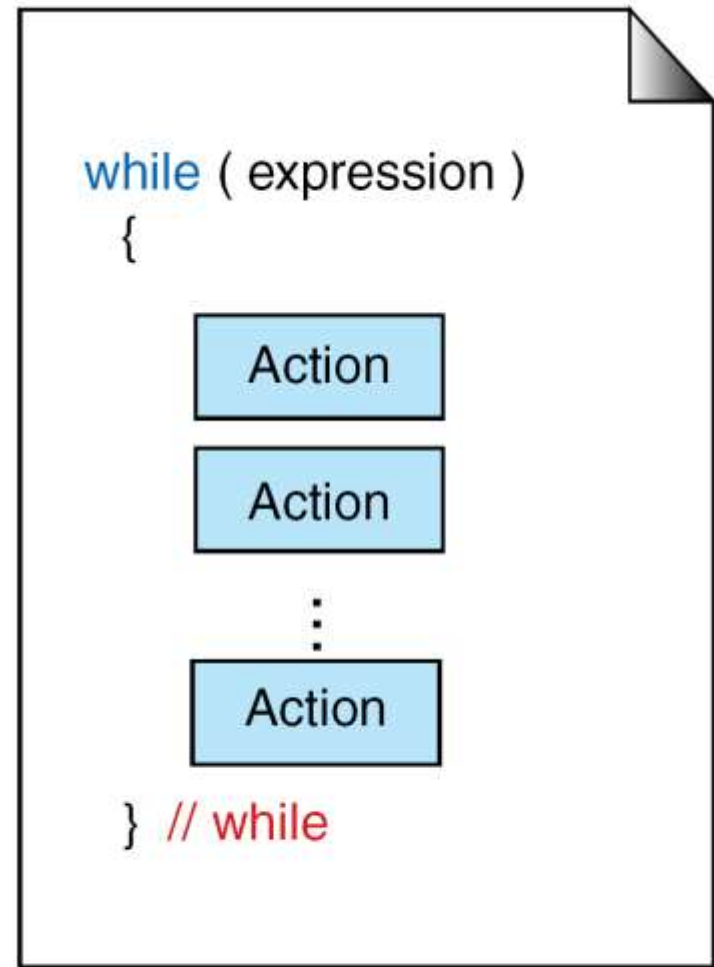
(a) Flowchart

```
while (expression)  
    statement
```

(b) Sample Code



(a) Flowchart



(b) C Language

PROGRAM 6-1 Process-control System Example

```
1  while (true)
2      {
3          temp = getTemperature();
4          if (temp < 68)
5              turnOnHeater();
6          else if (temp > 78)
7              turnOnAirCond();
8          else
9              {
10                 turnOffHeater();
11                 turnOffAirCond();
12             } // else
13     } // while true
```

PROGRAM 6-2 A *while* Loop to Print Numbers

```
1  /* Simple while loop that prints numbers 10 per line.
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6
7  int main (void)
8  {
9      // Local Declarations
10     int num;
11     int lineCount;
12
13     // Statements
14     printf ("Enter an integer between 1 and 100: ");
15     scanf ("%d", &num);          // Initialization
16
17     // Test number
18     if (num > 100)
19         num = 100;
20
```

PROGRAM 6-2 A *while* Loop to Print Numbers

```
21     lineCount = 0;
22     while (num > 0)
23     {
24         if (lineCount < 10)
25             lineCount++;
26         else
27         {
28             printf("\n");
29             lineCount = 1;
30         } // else
31         printf("%4d", num--);           // num-- updates loop
32     } // while
33     return 0;
34 } // main
```

Results:

Enter an integer between 1 and 100: 15

```
15  14  13  12  11  10   9   8   7   6
 5   4   3   2   1
```

PROGRAM 6-3 Adding a List of Numbers

```
1  /* Add a list of integers from the standard input unit
2     Written by:
3     Date:
4  */
5  #include <stdio.h>
6  int main (void)
7  {
8  // Local Declarations
9     int x;
10    int sum = 0;
11
12 // Statements
13    printf("Enter your numbers: <EOF> to stop.\n");
14    while (scanf("%d", &x) != EOF)
15        sum += x;
16    printf ("\nThe total is: %d\n", sum);
17    return 0;
18 } // main
```

PROGRAM 6-3 Adding a List of Numbers

Results:

Enter your numbers: <EOF> to stop

15

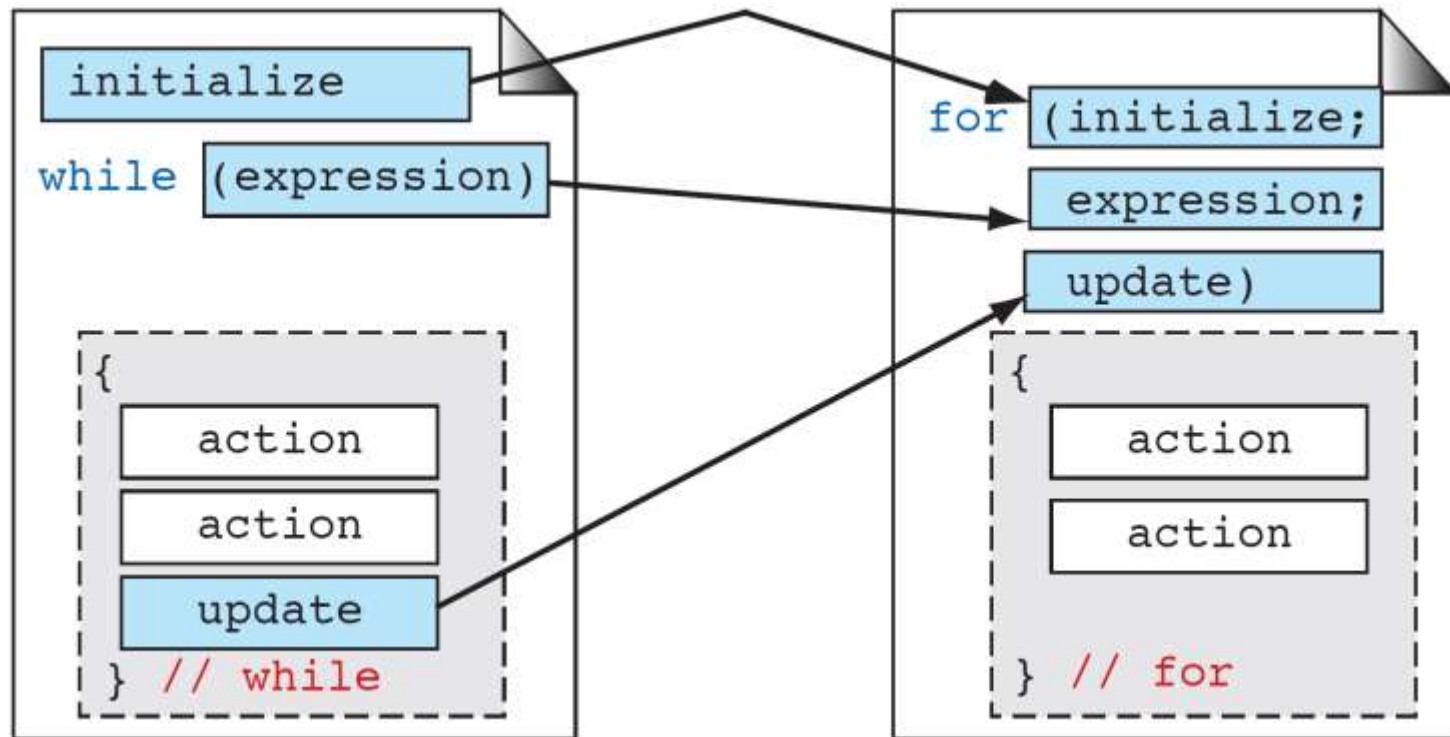
22

3^d

The total is: 40

Note

A *for* loop is used when a loop is to be executed a known number of times. We can do the same thing with a *while* loop, but the *for* loop is easier to read and more natural for counting loops.



PROGRAM 6-4 Example of a *for* Loop

```
1  /* Print number series from 1 to user-specified limit.
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6  int main (void)
7  {
8  // Local Declarations
9      int limit;
10
11 // Statements
12     printf ("\nPlease enter the limit: ");
13     scanf ("%d", &limit);
14     for (int i = 1; i <= limit; i++)
15         printf("\t%d\n", i);
16     return 0;
17 } // main
```


PROGRAM 6-4 Example of a *for* Loop

Results:

Please enter the limit: 3

1

2

3

PROGRAM 6-5 A Simple Nested *for* Loop

```
1  /* Print numbers on a line.  
2      Written by:  
3      Date:  
4  */  
5  #include <stdio.h>  
6  
7  int main (void)  
8  {  
9      // Statements  
10     for (int i = 1; i <= 3; i++)  
11     {  
12         printf("Row %d: ", i);  
13         for (int j = 1; j<= 5; j++)  
14             printf("%3d", j);  
15         printf("\n");  
16     } // for i  
17     return 0;  
18 } // main
```

PROGRAM 6-5 A Simple Nested *for* Loop

Results:

Row 1: 1 2 3 4 5

Row 2: 1 2 3 4 5

Row 3: 1 2 3 4 5

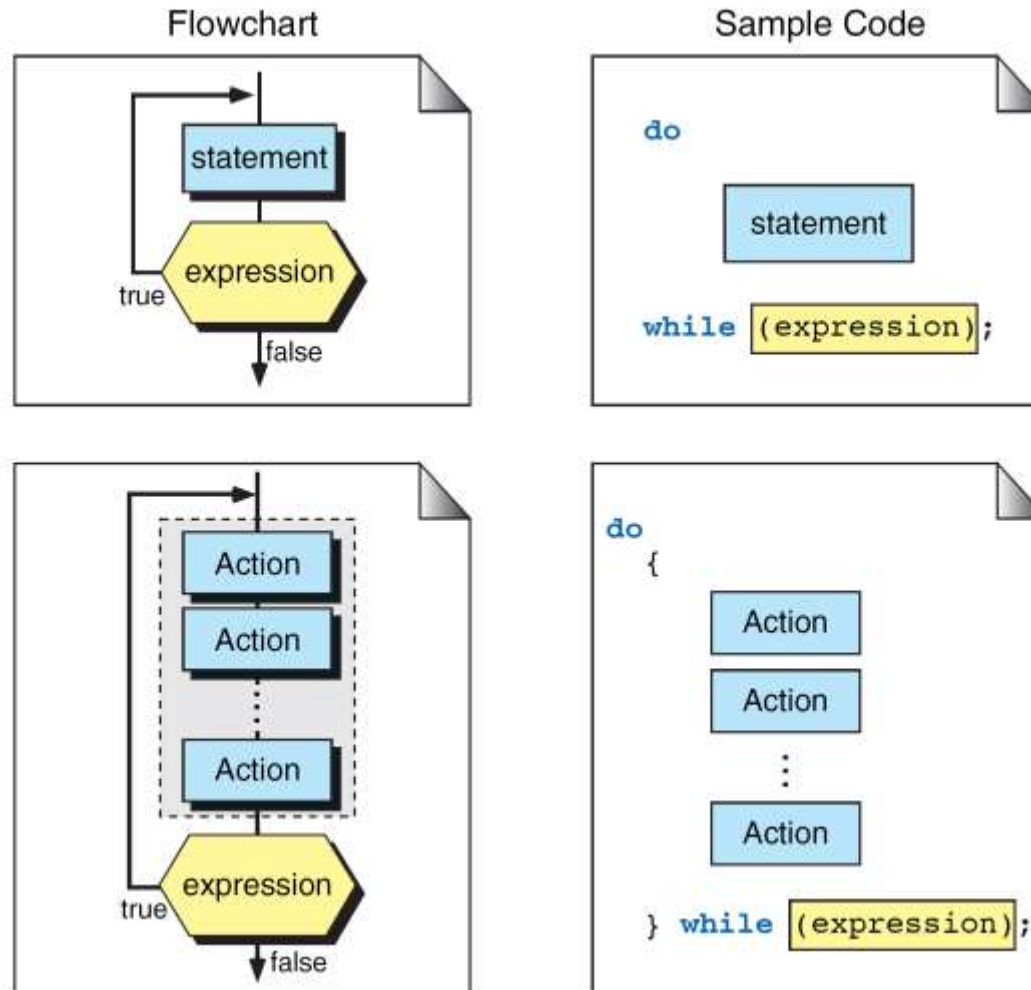


FIGURE 6-15 *do...while* Statement

PROGRAM 6-6 Two Simple Loops

```
1  /* Demonstrate while and do...while loops.
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6
7  int main (void)
8  {
9      // Local Declarations
10     int loopCount;
11
12     // Statements
13
14     loopCount = 5;
15     printf("while loop      : ");
16     while (loopCount > 0)
17         printf ("%3d", loopCount--);
18     printf ("\n\n");
```

PROGRAM 6-6 Two Simple Loops

```
19     loopCount = 5;
20     printf("do...while loop: ");
21     do
22         printf ("%3d", loopCount--);
23     while (loopCount > 0);
24     printf("\n");
25     return 0;
26 } // main
```

Results

```
while loop      :    5    4    3    2    1
```

```
do...while loop:    5    4    3    2    1
```

Pretest
nothing prints

```
while (false)
{
    printf("Hello World");
} // while
```

```
do
{
    printf("Hello World");
} while (false);
```

Post-test
Hello... prints

PROGRAM 6-7 Adding a List with the *do...while*

```
1  /* Add a list of integers from the standard input unit
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6
7  int main (void)
8  {
9      // Local Declarations
10     int x;
11     int sum = 0;
12     int testEOF;
13
14     // Statements
15     printf("Enter your numbers: <EOF> to stop.\n");
16     do
17     {
18         testEOF = scanf("%d", &x);
```


PROGRAM 6-7 Adding a List with the *do...while*

```
19         if (testEOF != EOF)
20             sum += x;
21     } while (testEOF != EOF);
22     printf ("\nTotal: %d\n", sum);
23     return 0;
24 } // main
```

Results:

Run 1:

Enter your numbers: <EOF> to stop.

10 15 20 25 ^d

Total: 70

Run 2:

Enter your numbers: <EOF> to stop.

^d

Total: 0

The diagram consists of two nested rectangular boxes. The inner box is light blue and contains the text "expression , expression". The outer box is a darker blue and contains the text "expression , expression" in addition to the inner box. This visualizes a list of expressions separated by commas, where the first two are grouped together.

expression , expression , expression

PROGRAM 6-8 Comparison of *while* and *do...while*

```
1  /* Demonstrate while and do..while loops.
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6
7  int main (void)
8  {
9      // Local Declarations
10     int loopCount;
11     int testCount;
12
13     // Statements
14     loopCount = 1;
15     testCount = 0;
16     printf("while loop:          ");
17     while (testCount++, loopCount <= 10)
18         printf("%3d", loopCount++);
19     printf("Loop Count:          %3d\n", loopCount);
```

PROGRAM 6-8 Comparison of *while* and *do...while*

```
20     printf("Number of tests:  %3d\n", testCount);
21
22     loopCount = 1;
23     testCount = 0;
24     printf("\ndo...while loop:  ");
25     do
26         printf("%3d",    loopCount++);
27     while (testCount++,  loopCount <= 10);
28
29     printf("\nLoop Count:      %3d\n", loopCount);
30     printf("Number of tests:  %3d\n", testCount);
31     return 0;
32 } // main
```

Results:

while loop: 1 2 3 4 5 6 7 8 9 10

Loop Count: 11

Number of tests: 11

do...while loop: 1 2 3 4 5 6 7 8 9 10

Loop Count: 11

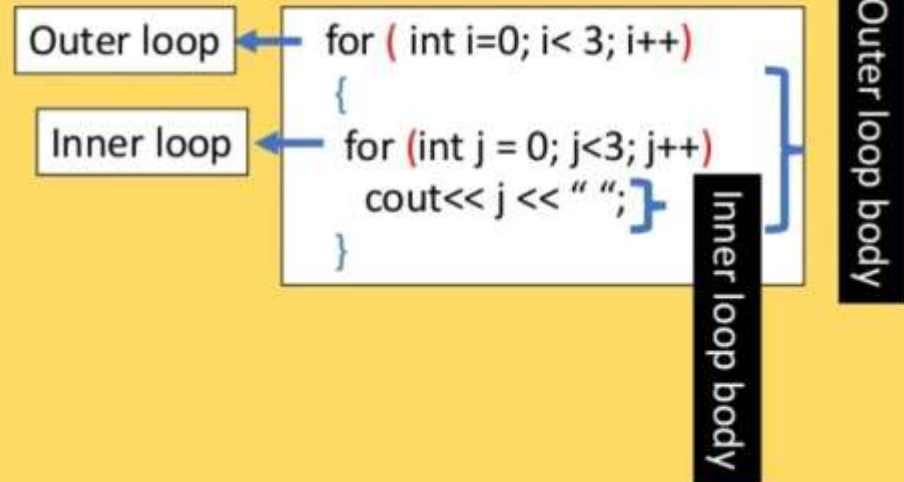
Number of tests: 10

Nested Loops

- What is a nested loop?
- Types of nested loops
- Working of a nested loop
- Example Programs

What are nested loops?

- A nested loop is a loop inside the body of another loop.
- The nested loop is known as the **inner loop** and the loop in which it is nested is known as the **outer loop**



Types of Nested Loops

```
graph TD; A[Types of Nested Loops] --> B[Nested for]; A --> C[Nested while]; A --> D[Nested do while]; A --> E[Mixed];
```

Nested for

Nested while

Nested do while

Mixed

Nested for loop

```
for ( initialization; test; update) ← outer loop
{ //outer loop statements;
  for( initialization; test; update) ← inner loop
  {
    //inner loop statements;
  }
}
```


Nested while loop

```
while ( test) ← outer loop
{ //outer loop statements;
  while( test) ← inner loop
  {
    //inner loop statements;
  }
}
```

Nested do while loop

```
do    ← outer loop
{    //outer loop statements;
do    ← inner loop
{
    //inner loop statements;
} while (test);
} while(test);
```

Working of nested loops: 1

- For each iteration of the outer loop, the inner loop is completely executed. So for example for the following next loop:

```
for(int i=0; i<3;i++)  
for(int j=0; j<3;j++)  
printf("%d ", i)
```

Working of nested loops: 1

- The statement `printf("%d ", i)` will execute $3 \times 3 = 9$ times. For each iteration of outer loop ($i = 0, 1, 2$), inner loop will execute 3 times ($j = 0, 1, 2$).
- Also note that there are no brackets. This is because both loops contain single statement; outer loop contains inner loop and inner loop contains single `printf` statement

Working of nested loops: 2

- If the loop contains multiple statements then there is a need for brackets. For the following nested loops :

```
for(int i=0; i<2;i++)  
{  
    printf("\n");  
    for(int j=0; j<3;j++)  
        printf("%d ", j);  
}
```

Working of nested loops: 2

- The statements `printf("%d ", j)`, will execute $2 \times 3 = 6$ times. For each iteration of outer loop ($i = 0, 1$), inner loop will execute three times ($j = 0, 1, 2$).
- There are brackets in the above nested loop . this is because the outer loop contains 2 statements; one `printf("\n")` and the inner loop. The inner loop contains single `printf` statement thus there are no brackets for inner loop.

Working of Nested Loops

```
1      2      8
for(int i= 0 ; i< 2; i++)
{
3
  cout<<"\n";
4      5      7
  for(int j = 0; j<2; j++)
6
    cout<<j <<" ";
}
```

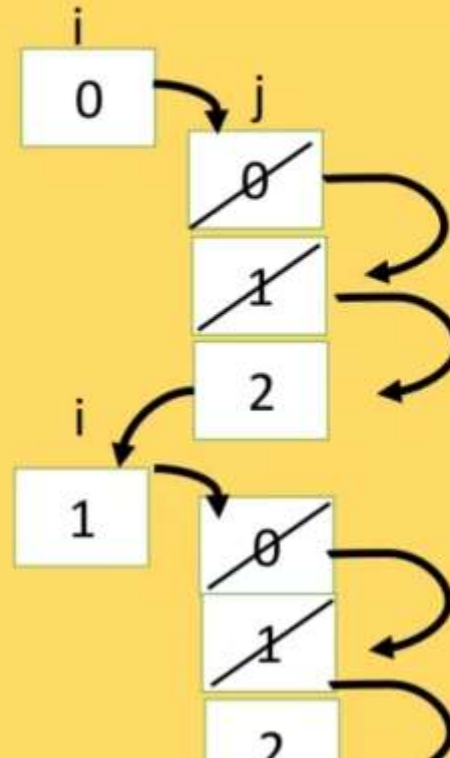
1. STEP 1: The outer loop is initialized with value of i as 0
2. STEP 2: Value of i is tested, since the condition is true (i<2), the loop is entered
3. STEP 3: A newline is displayed (cout<<"\n");. This is part of outer loop.
4. STEP 4: The control goes to inner loop, where j is initialized with 0
5. STEP 5: Value of j is tested, j<2 is true, inner loop is entered
6. STEP 6: The statement cout << j << "" is executed, value of j is displayed
7. STEP 7: The value of j is incremented.
8. Now STEPs 5, 6 and 7 are repeated till the condition j<2, becomes false.
9. When value of j is 2, control comes out of inner loop.
8. STEP 8: Now the control goes to outer loop update statement, i is incremented.
9. STEPs 2 – 7 are repeated. The steps are repeated for value of i = 1. This continues till value of i becomes 2. Then the outer loop is terminated.

Working of nested loops

```

  1      2      8
for(int i= 0 ; i< 2; i++)
{
  3
  cout<<"\n";
      4      5      7
  for(int j = 0; j<2; j++)
      6
    cout<<j << " ";
}

```



Output:

0 1
0 1

Nested Control Structures

- Suppose we want to create the following pattern

★

★ ★

★ ★ ★

★ ★ ★ ★

★ ★ ★ ★ ★

- In the first line, we want to print one star, in the second line two stars and so on

Nested Control Structures (continued)

- Since five lines are to be printed, we start with the following for statement

```
for (i = 1; i <= 5 ; i++)
```

- The value of `i` in the first iteration is 1, in the second iteration it is 2, and so on
- Can use the value of `i` as limit condition in another for loop nested within this loop to control the number of starts in a line

Nested Control Structures (continued)

- The syntax is:

```
int main()
{
    for (int i = 1; i <= 5 ; i++)
    {
        for (int j = 1; j <= i; j++)
            printf("*");
        printf("\n");
    }
}
```

Nested Control Structures (continued)

- What pattern does the code produce if we replace the first for statement with the following?

```
for (i = 5; i >= 1; i--)
```

- Answer:

```
*****
```

```
****
```

```
***
```

```
**
```

```
*
```

Nested Control Structures (continued)

- The syntax is:

```
int main()
{
    for (int i = 5; i >= 1 ; i--)
    {
        for (int j = 1; j <= i; j++)
            printf("*");
        printf("\n");
    }
}
```

Nested for loop exercise

- What is the output of the following nested for loop?

```
for (int i = 1; i <= 6; i++) {  
    for (int j = 1; j <= i; j++) {  
        printf("%d", i);  
    }  
    printf("\n");  
}
```

- Output?

Nested for loop exercise

- Output

1

22

333

4444

55555

Nested `for` loop exercise

- Create a nested `for` loops produce the following output.

```
....1
...22
..333
.4444
55555
```

Nested for loop exercise

- A `for` loop can have more than one loop nested in it.
- What is the output of the following nested `for` loops?

```
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= (5 - i); j++) {  
        printf(" ");  
    }  
    for (int k = 1; k <= i; k++) {  
        printf("%d", i);  
    }  
    printf("\n");  
}
```

Nested for loop exercise

- Answer:

```
1
22
333
4444
55555
```

Common nested loop bugs

- It is a common bug to accidentally type the wrong loop counter variable, which can lead to incorrect behavior.

- What is the output of the following nested loops?

```
for (int i = 1; i <= 10; i++) {  
    for (int j = 1; i <= 5; j++) {  
        printf("%d", j);  
    }  
    printf("\n");  
}
```

Common nested loop bugs

- Output

```
1234567891011121314151617181920212223242526272829303132333435363738394041
4243444546474849505152535455565758596061626364656667686970717273747576777
8798081828384858687888990919293949596979899100101102103104105106107108109
1101111121131141151161171181191201211221231241251261271281291301311321331
3413513613713813914014114214314414514614714814915015115215315415515615715
8159160161162163164165166167168169170171172173174175176177178179180181182
183184185186187188189190191192193194195196197198199200201202.....
```

Common nested loop bugs

- What is the output of the following nested loops?

```
for (int i = 1; i <= 10; i++) {  
    for (int j = 1; j <= 5; i++) {  
        printf("%d", j);  
    }  
    printf("\n");  
}
```


How to comment: for loops

- Place a comment on complex loops explaining *what* they do from a conceptual standpoint, not the mechanics of the syntax.

- Bad:

```
// This loop repeats 10 times, with i from 1 to 10.
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= 5; j++) { // loop goes 5
times
        printf("%d", j); // print the j
    }
    printf("\n");
}
```


How to comment: for loops

- Better:

// Prints 12345 ten times on ten separate lines.

```
for (int i = 1; i <= 10; i++) {  
    for (int j = 1; j <= 5; j++) {  
        printf("%d", j);  
    }  
    printf("\n"); // end the line of output
```

